

**LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK**

**MODUL 3
DESIGN PRINCIPLE (S.O.L.I.D)**

**DISUSUN OLEH :
BAGUS DWI PUTRA SETIAWAN - 2350081042**



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2025**

DAFTAR ISI

DAFTAR ISI.....	0
BAB I. HASIL PRAKTIKUM.....	1
I.1 Latihan ISP (Interface Segregation Principle).	1
I.1.A Langkah Kerja / Source Code	1
I.1.B Analisis.....	4
I.2 Latihan DPI (Dependency Inversion).....	5
I.2.A Langkah Kerja / Source Code	5
I.2.B Analisis.....	8
BAB II. TUGAS PRAKTIKUM	9
II.1 Tugas	9
BAB III. KESIMPULAN.....	11

BAB I. HASIL PRAKTIKUM

I.1 Latihan ISP (Interface Segregation Principle).

I.1.A Langkah Kerja / Source Code

Code Tanpa Prinsip ISP

Class VehicleInterface

```
interface VehicleInterface {  
    void drive();  
    void stop();  
    void refuel();  
    void openDoors();  
}
```

Class Motorcycle

```
class Motorcycle implements VehicleInterface {  
  
    // Can be implemented  
    @Override  
    public void drive() {  
    }  
  
    @Override  
    public void stop() {  
    }  
  
    @Override  
    public void refuel() {  
    }  
}
```

```
        // Cannot be implemented
        @Override
        public void openDoors() {
        }
    }
```

Class Car

```
class Car implements VehicleInterface {

    // Can be implemented
    @Override
    public void drive() {
    }

    @Override
    public void stop() {
    }

    @Override
    public void refuel() {
    }

    // Can be implemented
    @Override
    public void openDoors() {
    }
}
```

Code Dengan Prinsip ISP

Class Interface VehicleInterface

```
interface VehicleInterface {  
    void drive();  
    void stop();  
    void refuel();  
}
```

Class Interface DoorInterface

```
interface DoorInterface {  
    void openDoors();  
}
```

Class Motorcycle

```
class Motorcycle implements VehicleInterface {  
    @Override  
    public void drive() {}  
  
    @Override  
    public void stop() {}  
  
    @Override  
    public void refuel() {}  
}
```

Class Car

```
class Car implements VehicleInterface,  
DoorInterface {  
    @Override  
    public void drive() {}  
  
    @Override  
    public void stop() {}  
  
    @Override  
    public void refuel() {}  
  
    @Override  
    public void openDoors() {}  
}
```

I.1.B Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu menerapkan Interface Segregation Principle (ISP) untuk memastikan bahwa setiap interface hanya berisi metode yang benar-benar diperlukan oleh kelas yang mengimplementasikannya. Pada kode awal, interface VehicleInterface memiliki metode openDoors(), yang tidak relevan bagi kelas Motorcycle. Hal ini melanggar prinsip ISP karena kelas tersebut dipaksa mengimplementasikan metode yang tidak digunakannya.

Setelah penerapan prinsip ISP, interface dipecah menjadi dua yaitu VehicleInterface untuk fungsi umum kendaraan seperti drive(), stop(), dan refuel(), serta DoorInterface untuk kendaraan yang memiliki pintu. Dengan pemisahan ini, Car dapat mengimplementasikan kedua interface tersebut, sedangkan Motorcycle hanya mengimplementasikan yang relevan.

I.2 Latihan DPI (Depedency Inversion).

I.2.A Langkah Kerja / Source Code

Code Tanpa Prinsip DPI

Class Car

```
class Car {  
    private final Engine engine;  
  
    public Car(Engine engine) {  
        this.engine = engine;  
    }  
  
    void start() {  
        engine.start();  
    }  
}
```

Class Engine

```
class Engine {  
    void start() {  
    }  
}
```

Class DieselEngine

```
class DieselEngine {  
    void start() {  
    }  
}
```

Code Dengan Prinsip DPI

Class Car

```
class Car {  
    private final EngineInterface engine;  
  
    public Car(EngineInterface engine) {  
        this.engine = engine;  
    }  
  
    void start() {  
        engine.start();  
    }  
}
```

Class Interface EngineInterface

```
interface EngineInterface {  
    void start();  
}
```

Class PetrolEngine

```
class PetrolEngine implements EngineInterface {  
    @Override  
    public void start() {  
    }  
}
```


Class HybridEngine

```
class HybridEngine implements EngineInterface {  
    @Override  
    public void start() {  
    }  
}
```

Class DieselEngine

```
class DieselEngine implements EngineInterface {  
    @Override  
    public void start() {  
    }  
}
```

Class Main

```
class Main {  
    public static void main(String[] args) {  
        Car fuelCar = new Car(new PetrolEngine());  
        Car dieselCar = new Car(new  
DieselEngine());  
        Car hybridCar = new Car(new  
HybridEngine());  
  
        // Boom boom  
        fuelCar.start();  
        dieselCar.start();  
        hybridCar.start();  
    }  
}
```

```
}
```

I.2.B Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu menerapkan Dependency Inversion Principle (DPI) yang menekankan agar kelas tingkat tinggi tidak bergantung langsung pada kelas konkret, melainkan pada abstraksi seperti interface. Pada kode awal, kelas Car bergantung langsung pada kelas Engine, sehingga bila jenis mesin berubah, kode di dalam Car juga harus dimodifikasi. Ketergantungan semacam ini melanggar prinsip DPI karena menurunkan fleksibilitas dan memperumit pengembangan sistem.

Dalam versi yang telah diperbaiki, dibuat interface EngineInterface sebagai lapisan abstraksi yang diimplementasikan oleh berbagai jenis mesin seperti PetrolEngine, DieselEngine, dan HybridEngine. Kelas Car kini hanya bergantung pada EngineInterface, bukan pada kelas mesin tertentu.

BAB II. TUGAS PRAKTIKUM

II.1 Tugas

1. Pada Latihan 4.2.1 ISP, Jelaskan alasan kenapa bentuk/design codenya harus diubah?

Jawab : Desain kode harus diubah karena melanggar prinsip Interface Segregation Principle (ISP). Pada kode awal, satu interface VehicleInterface memiliki terlalu banyak metode yang tidak relevan bagi semua kelas. Misalnya, Motorcycle dipaksa mengimplementasikan metode openDoors() padahal tidak memiliki pintu. Oleh karena itu, interface perlu dipecah agar setiap kelas hanya mengimplementasikan metode yang sesuai dengan kebutuhannya.

2. Ubah Class Motorcycle menggunakan fungsi Java !

Jawab :

```
class Motorcycle implements VehicleInterface {  
    @Override  
    public void drive() {  
        System.out.println("Motorcycle is  
driving...");  
    }  
  
    @Override  
    public void stop() {  
        System.out.println("Motorcycle has  
stopped.");  
    }  
  
    @Override  
    public void refuel() {  
        System.out.println("Motorcycle is  
refueling.");  
    }  
}
```

```
}  
}
```

3. Latihan 4.2.2 DPI, Jelaskan alasan kenapa bentuk/design codenya harus diubah?

Jawab : Desain awal melanggar Dependency Inversion Principle (DPI) karena kelas Car bergantung langsung pada kelas konkret Engine. Hal ini menyebabkan ketergantungan yang kuat dan sulit dikembangkan. Dengan membuat EngineInterface, Car bergantung pada abstraksi, bukan pada implementasi. Hal ini membuat lebih fleksibel, mudah diuji, dan memungkinkan penambahan jenis mesin baru tanpa mengubah kode di kelas Car.

BAB III. KESIMPULAN

Kesimpulan yang dapat saya ambil dalam pertemuan ketiga pada Praktikum Implementasi Perangkat Lunak adalah dapat memahami dan mempraktikkan penerapan Design Principle khususnya prinsip-prinsip pada konsep S.O.L.I.D seperti Interface Segregation Principle (ISP) dan Dependency Inversion Principle (DPI). Melalui latihan ISP, mempelajari untuk memisahkan interface agar setiap kelas hanya memiliki tanggung jawab terhadap metode yang relevan. Sementara itu, pada latihan DPI, memahami pentingnya membalik ketergantungan dengan menggunakan abstraksi (interface) sehingga kode menjadi lebih fleksibel, mudah dikembangkan, dan tidak saling bergantung antar kelas.